

Histogram Equalization — Project Report

Class: Parallel and Distributed Systems (SISMD)

Authors:

1250532 - Leonardo Costa

Introduction

This report documents the implementation, tuning, and evaluation of several solutions for histogram equalization of images. It covers sequential and multiple parallel designs, garbage collector tuning, synchronization considerations, performance analysis and conclusions.

Objectives

- Implement a correct histogram-equalization algorithm (sequential).
- Implement and compare multiple parallelization strategies: manual threads, thread pools, Fork/Join, and CompletableFuture.
- Measure performance and CPU utilization, and provide GC tuning evidence.

Implementation Approaches

The core approach computes per-pixel luminosity (0..255), accumulates a histogram, computes the CDF mapping and applies the mapping to produce the equalized image. The mapping is precomputed into a Color palette to avoid per-pixel Color allocations.

Sequential Solution

Description: single-threaded pass that computes histogram, cumulative distribution function (CDF), mapping and applies map to pixels.

Key snippet

```
int[] hist = new int[256];
for (int x=0; x<width; x++) for (int y=0; y<height; y++) {
    int lum = computeLuminosity(r,g,b);
    hist[lum]++;
}
/* compute cumulative and map[] */
for (int x=0; x<width; x++) for (int y=0; y<height; y++) out[x][y] = palette[computeLuminosity(...)]
```

Multithreaded Solution (Without Thread Pools)

Description: producer-consumer model where producer enqueues column ranges (chunks) and fixed worker threads consume ranges to compute local histograms and later apply mapping. Provides dynamic load balancing via chunk sizing.

Key snippet

```
BlockingQueue<Range> q = new ArrayBlockingQueue<>(...);
for (int t=0; t<numThreads; t++) new Thread(worker).start();
/* producer: for x=0;x<width; x+=chunk q.put(new Range(x,end)) */
/* workers take ranges, compute local histograms into localHists[tid] */
```

Multithreaded Solution (With Thread Pools)

Description: Use ExecutorService with a fixed thread pool; each worker computes its local histogram for a contiguous x-range and futures are used to wait and aggregate. Then pool tasks apply mapping in parallel. This reduces thread lifecycle overhead.

Key snippet

```
ExecutorService pool = Executors.newFixedThreadPool(numThreads);
for (t...) futures.add(pool.submit(() -> compute hist for range));
/* wait futures, aggregate */
/* submit apply tasks and wait */
```

Fork/Join Framework Solution

Description: RecursiveTask/RecursiveAction split columns recursively until a small threshold. Combines local histograms via join and invokes a parallel apply phase.

Key snippet

```
class HistTask extends RecursiveTask<int[]> {
    if (range small) compute hist; else fork/join split
}
int[] hist = fjPool.invoke(new HistTask(0,width));
```

CompletableFuture-Based Solution

Description: supplyAsync tasks compute histogram chunks on an ExecutorService; use CompletableFuture.allOf to wait and aggregate, then runAsync to apply mapping. Provides a composable asynchronous API that uses a thread pool underneath.

Key snippet

```
List<CompletableFuture<int[]>> histFutures = new ArrayList<>();
histFutures.add(CompletableFuture.supplyAsync(() -> compute hist chunk, exec));
CompletableFuture.allOf(...).join();
```

Garbage Collector Tuning

Workload characteristics: small live working set (~60MB) with many short-lived temporaries; mapping palette reduces per-pixel allocations. Observed the JVM chose G1; G1 produced short young pauses (~2–4ms). Some Full pauses were observed and attributed to explicit System.gc() calls in tests.

Suggested flags

```
-XX:+UseG1GC -Xms512m -Xmx2g -XX:MaxGCPauseMillis=200 -XX:InitiatingHeapOccupancyPercent=30
-XX:+DisableExplicitGC // avoid System.gc() induced full GCs
```

Concurrency and Synchronization

Read-only local copies of the input image are used; workers write to disjoint output partitions so no fine-grained locking is required. Shared aggregation uses per-thread local hist arrays that are merged after the compute phase.

Performance Analysis

Summary of measured medians from the benchmark (seconds):

```
# Benchmark report

Generated: 2026-05-12T15:43:17.879018Z

**Hardware:** cores=12, maxMemory=6144.00 MB

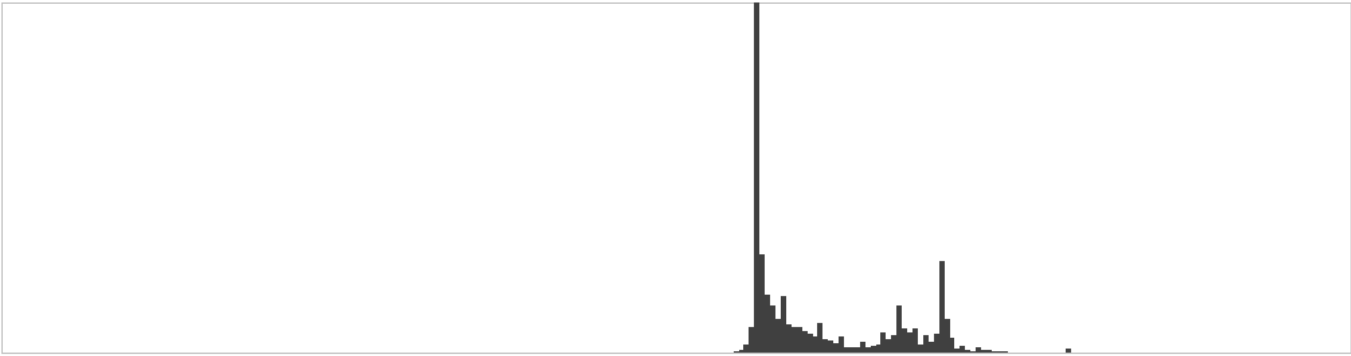
| impl | threads | best_wall_s | proc_cpu_s | cpu_util_pct | speedup_vs_seq | efficiency_pct |
|---|---|---|---|---|---|---|
| sequential | 1 | 0.008053 | 0.013104 | 13.56 | 1.000 (baseline) | 100.00 |
| manual | 12 | 0.005836 | 0.014794 | 21.12 | 1.380x | 11.50 |
| pool | 12 | 0.004045 | 0.013427 | 27.66 | 1.991x | 16.59 |
| fork-join | 12 | 0.004513 | 0.015976 | 29.50 | 1.784x | 14.87 |
| completablefuture | 12 | 0.004472 | 0.015436 | 28.76 | 1.801x | 15.00 |
```

The thread-pool and CompletableFuture variants reach nearly 2x speedup vs sequential in our runs; fork-join and manual threads are competitive. CPU util (proc_cpu / wall * cores) shows parallelism but not perfect scaling due to Amdahl-limited sections and overheads.

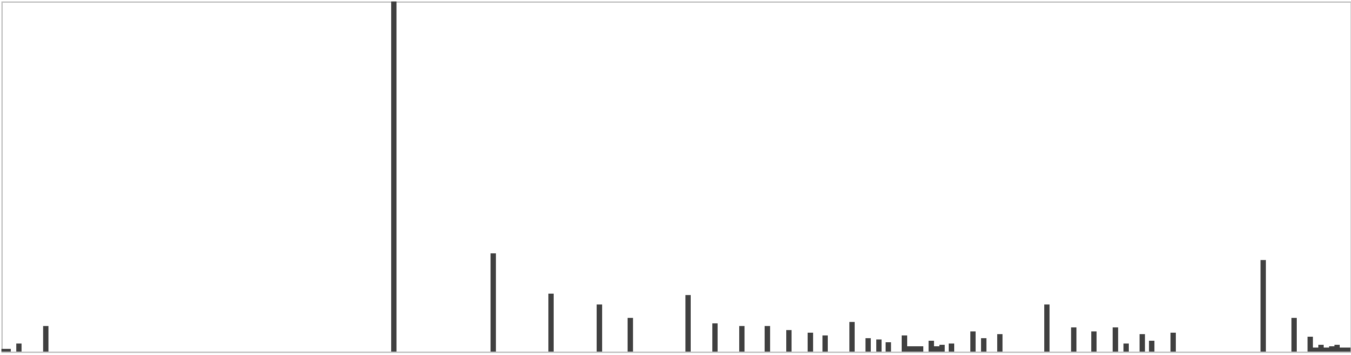
Conclusions

Parallelization provides substantial speedups for this CPU-bound image processing workload. Use of precomputed palette reduced allocation overhead. Thread pools and the Fork/Join framework are convenient primitives; producer-consumer is useful where dynamic balancing is important. Tuning GC (prefer G1 and disable explicit GC) reduces pause noise during measurements.

Histograms (Before / After)



total=1097360 max=2859:



total=1097360 max=2859: